WIPRO CAPSTONE PROJECT – MODULE 6

In-Vehicle Infotainment (IVI) Systems

Simulating Message Flow between the Media Service and the Audio HAL

How a user input propagates through an Android Automotive audio stack

Student Name	Lisa Mondal
Enrollment No.	12023002028133
Registration No.	304202300901051
Programme	B.Tech, Computer Science and Engineering (Artificial Intelligence and Machine Learning)
Institute	Institute of Engineering and Management, Newtown
Project Option	Option 3 – Media Service to Audio HAL message-flow simulation
Tools	Python, Pandas, NumPy, Matplotlib, Google Colab
Dataset	Synthetic logcat-style IVI event log (400 user actions, 3,257 log lines)

Abstract

This project simulates the path a driver's touch or steering-wheel input takes through the audio stack of an Android-Automotive-style infotainment system. The path runs from the input dispatcher, through the media app, the MediaSession service and the car audio service, into AudioFlinger, the Audio HAL and finally the DSP and amplifier. A Python simulation generates logcat-style logs for 400 user actions (441 transactions in total, including 41 automatic restores after navigation prompts). The logs are parsed with a single regular expression into a structured dataset and analysed to show how long each layer takes, which actions are slowest and where failures occur.

Of 441 transactions, 98.4 % completed successfully (426 first time and 8 after a HAL retry). The Audio HAL is the largest contributor to latency at 35.2 % of processing time and the source of most warnings and errors. 99.5 % of clean transactions finished inside a 100 ms responsiveness budget. The latency and fault figures come from synthetic distributions and describe the simulation, not a measured head unit.

1. Introduction

An in-vehicle infotainment (IVI) system combines media playback, navigation, phone and projection features on one head unit. All of them share the same speakers, so a single press of a Play button involves several software components that must agree on who may play, at what volume and through which output bus. When these components exchange messages slowly or lose one, the driver notices a laggy button, a track that does not start or navigation prompts that do not lower the music.

The Module 6 brief offers four options. This report takes Option 3: simulate the message flow between the Media Service and the Audio HAL using logs and a flowchart, and demonstrate how user input propagates through the stack. The option is well suited to a data-driven treatment, because every hop in a real stack leaves a log line that can be parsed and measured.

2. Objectives

- Model the layers between a user input and the amplifier, with the interface and data exchanged at each hop.
- Generate logcat-style logs for common media actions (play, pause, next, previous, volume) and for a navigation prompt that interrupts media.
- Parse the logs into a structured dataset and reconstruct the flow of a single transaction as a sequence diagram.
- Measure per-layer and end-to-end latency, and count failures, to identify the main bottleneck.
- Show the interruption use case: a navigation prompt lowers (ducks) the media volume and media returns to normal volume automatically when the prompt ends.

3. System Architecture

The simulated stack has seven layers. Figure 1 shows the request path (solid arrows), the acknowledgement or callback path (dashed arrows), and the interface and data exchanged between neighbouring layers.

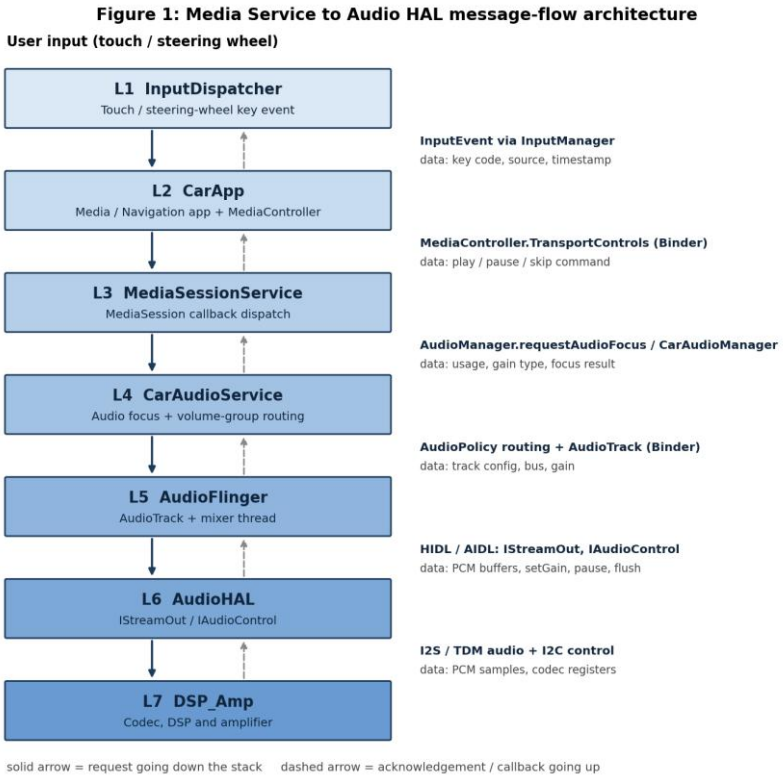


Figure 1: Media Service to Audio HAL message-flow architecture

Layer	Log tag	Role
L1	InputDispatcher	Receives the touch or steering-wheel key event and identifies the key code and source.
L2	CarApp	Media or navigation app; turns the click into a MediaController transport command.
L3	MediaSessionService	Delivers the command to the active MediaSession (for example the music app).
L4	CarAudioService	Handles audio focus and routes volume changes to the right volume group and bus.
L5	AudioFlinger	Creates, flushes and pauses AudioTracks and mixes streams.
L6	AudioHAL	Hardware abstraction (IStreamOut, IAudioControl): opens streams, writes PCM, sets gain.
L7	DSP_Amp	Codec, DSP and amplifier: unmute, gain change, buffer reset.

Volume actions skip L3, because volume goes straight from the UI to CarAudioService. A navigation prompt starts in the navigation app at L2 and also skips L3.

4. Methodology

4.1 Simulation design

Each user action becomes a transaction with an id. The action visits the layers on its path in order and each layer writes one log line containing the transaction id, the action, what the layer did and the time

it spent (dt). Layer processing times are drawn from log-normal distributions whose means are set per layer (for example 14 ms for the Audio HAL and 3 ms for the input dispatcher); a per-transaction load factor makes layers slow down or speed up together, and the HAL is scaled up for actions that open or flush a stream. A fixed random seed (42) makes every run reproducible.

Two faults are injected:

- **Audio focus denied.** About 3 % of PLAY requests are refused at CarAudioService because a higher-priority stream is active. The transaction stops at L4 and never reaches the HAL.
- **HAL write timeout.** About 3.5 % of transactions see the HAL take 120 to 250 ms. Sixty percent recover on a retry (status RETRY_OK); the rest fail and the stream is reset (status FAILED), so the DSP layer is never reached.

Every navigation prompt is followed 3 to 6 seconds later by an automatic NAV_RESTORE transaction, which models media returning to full volume.

4.2 Log format and parsing

Logs use the standard logcat layout so that the same parser would work on a real adb capture. One line looks like this:

```
09-29 09:00:15.933  1544  1544 I CarApp : [txn=0001] [NEXT] onNextClicked -> transportControls.skipToNext() | dt=6.73ms
```

A single regular expression extracts the timestamp, process and thread ids, level, tag, transaction id, action, message and dt. Each layer's tag maps to a layer id. Lines whose message starts with RESULT carry the final status and the end-to-end time of a transaction and are separated from the per-layer events.

4.3 Dataset

File	Rows	Contents
ivi_logcat.txt	3,257 lines	Raw logcat-style log, as generated by the simulation.
ivi_message_flow_events.csv	2,816	One row per layer event: ts, pid, tid, level, tag, txn, action, message, dt_ms, layer_id, status.
ivi_transactions.csv	441	One row per transaction: txn, action, ts, status, e2e_ms.
metrics.json	–	Summary numbers used in this report.

5. Message Flow of a User Input

Figure 2 is reconstructed from the log of one successful PLAY transaction (txn 0005). The tap becomes a key event, then a transport command, then an audio-focus request that is granted, then an AudioTrack, then an HAL stream open with the first buffer written, and finally the codec unmute and amplifier enable. Longer gaps on the diagram are the time a layer spends working (the HAL open and write is the longest step).

Figure 2: Sequence diagram of PLAY (txn 0005) reconstructed from logcat

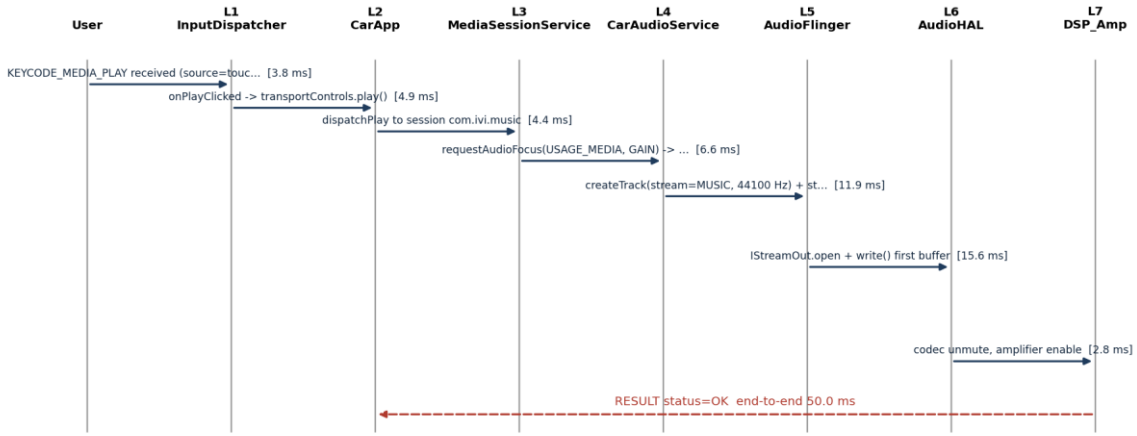


Figure 2: Sequence diagram of a PLAY transaction reconstructed from logcat

5.1 Interruption scenario: navigation prompt and automatic media resume

The first navigation prompt in the log and the restore that follows it are shown below. The prompt requests transient audio focus with ducking allowed, so media is not stopped: AudioFlinger halves the media track gain, the HAL is told the focus change and the DSP lowers the media bus by 6 dB. When the prompt finishes, focus is abandoned, the gain returns to 1.0 and the DSP returns to 0 dB, so music resumes without any user action.

```
09-29 09:00:37.081 I CarApp: [txn=0003] [NAV_PROMPT] NavigationApp voice prompt: 'turn left in 200 m' |
dt=11.93ms
09-29 09:00:37.093 I CarAudioService: [txn=0003] [NAV_PROMPT] requestAudioFocus(NAV_GUIDANCE,
GAIN_TRANSIENT_MAY_DUCK) -> media ducked | dt=5.18ms
09-29 09:00:37.098 I AudioFlinger: [txn=0003] [NAV_PROMPT] duck media track gain 1.0 -> 0.5 | dt=8.16ms
09-29 09:00:37.106 I AudioHAL: [txn=0003] [NAV_PROMPT] IAudioControl.onAudioFocusChange(DUCK) | dt=23.17ms
09-29 09:00:37.129 I DSP_Amp: [txn=0003] [NAV_PROMPT] DSP gain -6 dB on media bus | dt=3.28ms
09-29 09:00:37.132 I CarApp: [txn=0003] [NAV_PROMPT] RESULT status=OK e2e=51.71ms | dt=0.00ms
09-29 09:00:42.364 I CarApp: [txn=0004] [NAV_RESTORE] NavigationApp prompt finished | dt=8.28ms
09-29 09:00:42.372 I CarAudioService: [txn=0004] [NAV_RESTORE] abandonAudioFocus(NAV_GUIDANCE) -> media
resumes | dt=13.83ms
09-29 09:00:42.386 I AudioFlinger: [txn=0004] [NAV_RESTORE] restore media track gain 0.5 -> 1.0 | dt=11.49ms
09-29 09:00:42.398 I AudioHAL: [txn=0004] [NAV_RESTORE] IAudioControl.onAudioFocusChange(GAIN) | dt=29.16ms
09-29 09:00:42.427 I DSP_Amp: [txn=0004] [NAV_RESTORE] DSP gain 0 dB on media bus | dt=5.85ms
09-29 09:00:42.433 I CarApp: [txn=0004] [NAV_RESTORE] RESULT status=OK e2e=68.60ms | dt=0.00ms
```

6. Results

6.1 Outcomes

Outcome	Count	Share	Meaning
OK	426	96.6 %	Completed first time
RETRY_OK	8	1.8 %	HAL timeout, retry worked
FOCUS_DENIED	5	1.1 %	Stopped at CarAudioService
FAILED	2	0.5 %	HAL retry failed, stream reset

Table 1: Transaction outcomes (441 transactions)

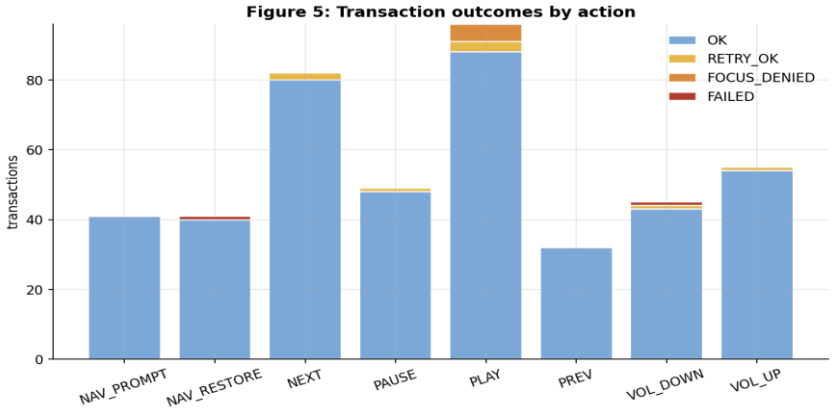


Figure 3: Transaction outcomes by action

Overall 98.4 % of transactions completed (434 of 441). WARN or ERROR lines came from only two layers: 5 at CarAudioService (all focus denials) and 12 at the Audio HAL (10 timeouts plus 2 failed retries).

6.2 End-to-end latency

Action	n	Mean (ms)	Median (ms)	P95 (ms)	Max (ms)
NAV_PROMPT	41	40.2	40.4	52.3	56.8
NAV_RESTORE	40	40.7	40.8	53.6	68.6
NEXT	80	52.3	51.0	74.3	103.0
PAUSE	48	47.1	46.9	65.9	82.0
PLAY	88	60.6	59.0	82.4	101.8
PREV	32	45.8	44.1	63.2	72.5
VOL_DOWN	43	41.1	39.2	58.1	74.2
VOL_UP	54	41.2	38.8	57.9	70.5

Table 2: End-to-end latency of transactions that completed first time

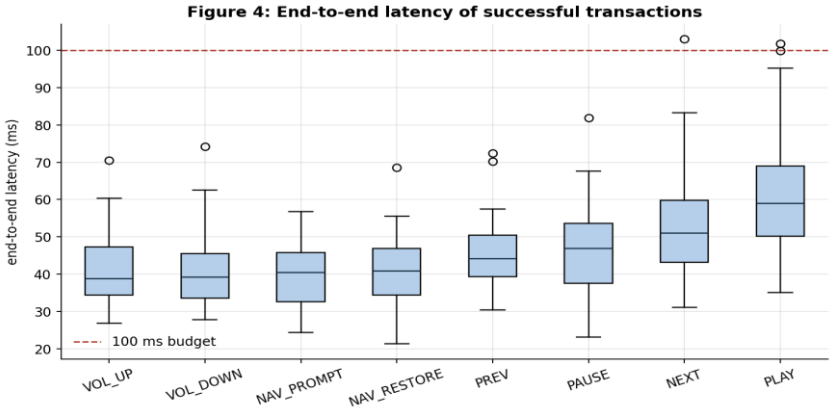


Figure 4: End-to-end latency per action against the 100 ms budget

PLAY is the slowest action (mean 60.6 ms) because it requests focus and opens a stream. NEXT and PLAY are the only actions that reach or pass 100 ms. Volume changes and navigation ducking are the fastest (about 40 ms), since they touch fewer layers. 99.5 % of first-time transactions stayed within

the 100 ms budget. Every one of the 10 timeout transactions exceeded it, because a timeout alone costs 120 to 250 ms.

6.3 Where the time goes

Layer	n	Mean	Median	P95	Max	Time share
L1 InputDispatcher	345	3.08	2.82	5.42	7.07	5.2 %
L2 CarApp	426	7.08	6.78	12.23	23.04	14.7 %
L3 MediaSessionService	248	4.58	4.24	8.26	13.68	5.5 %
L4 CarAudioService	426	6.42	6.10	10.42	17.68	13.3 %
L5 AudioFlinger	426	9.06	8.41	16.34	25.93	18.8 %
L6 AudioHAL	426	16.92	14.80	34.25	62.09	35.2 %
L7 DSP_Amp	426	3.49	3.28	6.11	8.80	7.3 %

Table 3: Per-layer processing time in ms (transactions that completed first time)

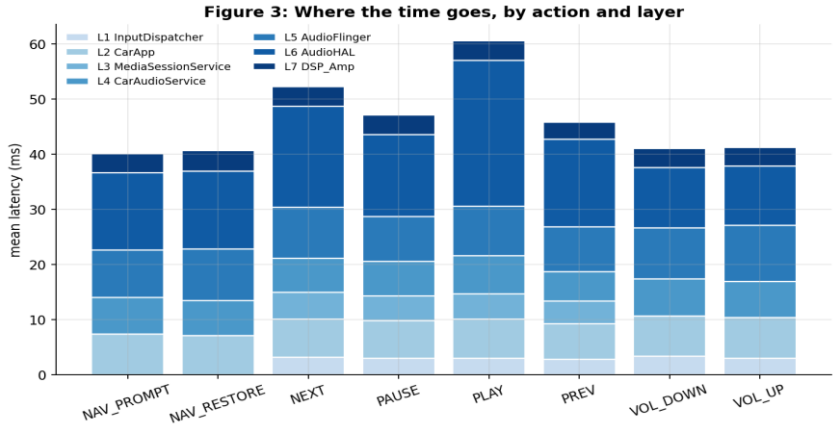


Figure 5: Mean time per layer for each action

The Audio HAL takes 35.2 % of all layer time, followed by AudioFlinger (18.8 %) and the app layer (14.7 %). The HAL also has the widest spread (median 14.8 ms, P95 34.3 ms, max 62.1 ms).

7. Discussion

- **The HAL is the bottleneck.** Because the HAL sits closest to hardware and does the heaviest work (opening streams, writing buffers), it dominates both average time and tail latency. Improving HAL open time would help PLAY more than tuning any upper layer.
- **Failures are concentrated and recoverable.** All faults occurred at two points: the focus decision and the HAL write. A retry recovered 8 of the 10 timeouts, which supports adding a bounded retry in the HAL client, together with logging that names the transaction.
- **Shared transaction ids make debugging easy.** Carrying one id through every layer let the log be regrouped into a sequence diagram with a few lines of code. Real stacks are harder to correlate unless the id is deliberately propagated.
- **Ducking is cheap and automatic.** The navigation prompt and its restore each took about 40 to 70 ms and needed no user action, which is the behaviour the brief asks for in an interruption use case.

8. Limitations

- Latencies, timeouts and focus denials are synthetic. The shape of the results follows the parameters chosen, so they show how the analysis works, not how a specific head unit performs.
- The layer names and interfaces follow the Android Automotive audio path in general terms. A real device may use different processes, extra layers (for example a vendor audio service) or a different HAL interface version.
- Only sequential single-user actions are simulated; overlapping actions, phone calls and multi-zone audio are not modelled.
- The 100 ms budget is an assumption made for this project.

The pipeline can be pointed at a real capture (for example an adb logcat of a Raspberry Pi or emulator run) by adding the same transaction id to each layer's messages and reusing the parser and charts.

9. Conclusion

The simulation shows how one press on an IVI screen becomes a chain of messages across seven layers, and how the logs of that chain can be parsed into a dataset that answers practical questions. In the 441 simulated transactions, 98.4 % completed, most finished within 100 ms, the Audio HAL accounted for about a third of processing time and most warnings, and a navigation prompt ducked and then restored media automatically. The same method, with a transaction id in every log line and a parser like the one provided, can be applied to real IVI logs to locate bottlenecks and failures.

Appendix A – Files and how to run

- Module6_IVI_MediaService_AudioHAL_Simulation.ipynb: upload to Google Colab and choose Runtime, Run all. It generates the log, parses it, draws all figures and offers a zip download of the outputs.
- Dataset files: ivi_logcat.txt, ivi_message_flow_events.csv and ivi_transactions.csv (also produced by the notebook, with the same seed).

Appendix B – References

- Android Open Source Project, Automotive audio and Audio HAL documentation, source.android.com.
- Android Developers, Manage audio focus and MediaSession documentation, developer.android.com.